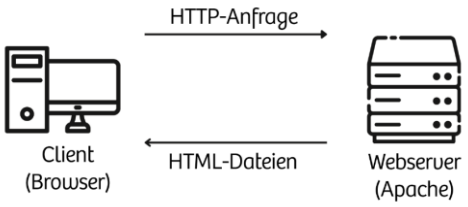
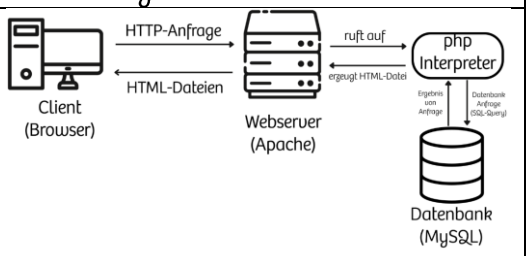
⚠ *Achtung*: Sie können dieses Dokument verwenden, um für Prüfungen zu lernen, aber ich übernehme keine Verantwortung, wenn irgendwo im Dokument Fehler enthalten sind. Durch die Verwendung dieses Dokuments akzeptieren Sie diese Bedingung. Wenn in Dokument Fehler sind oder etwas fehlt, bitte schreiben sie eine Nachricht mit dem Fehler in Ömers Dokumente Whatsapp Gruppe. ⚠

-Viel Spaß üben, Ömer

Informatik KA Vorbereitung 02.06.2025

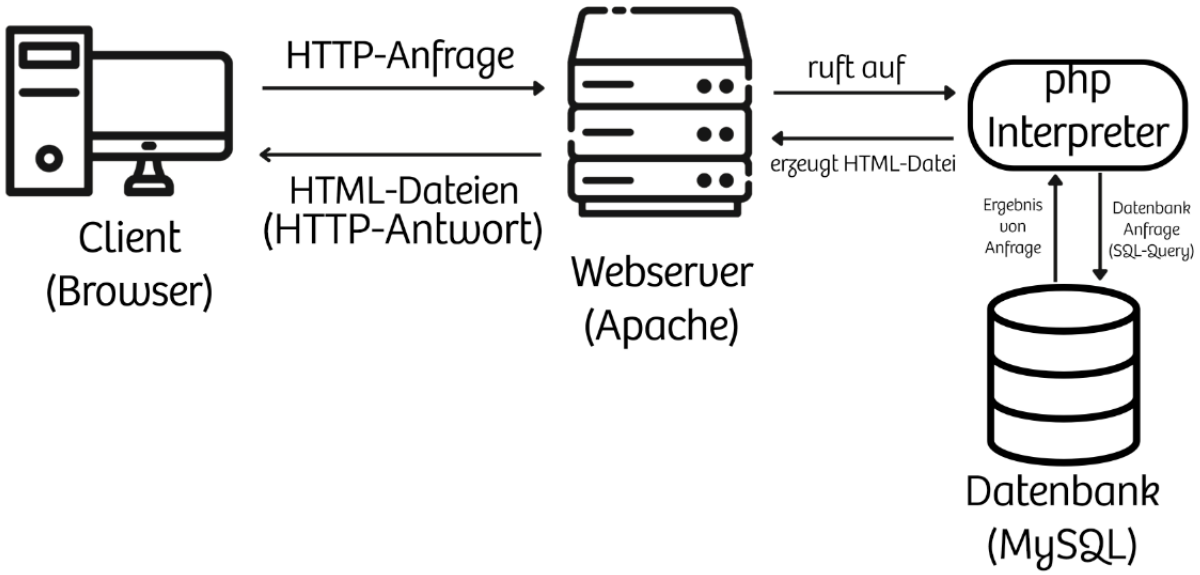
statische vs. dynamische Webseiten

Thema	Statische Webseite	Dynamische Webseite
Aufbau		
Datentypen	HTML, CSS, usw.	HTML, PHP, SQL (Datenbank), usw.
Inhalt	HTML-Code, der sich nicht ändert (immer gleich)	HTML-Code wird beim Aufruf erzeugt d. H.: es kann abhängig von Benutzer sein (kann sich ändern)
Vorteile	Schnell und einfach	Flexibel und personalisierbar
Nachteile	Keine Interaktivität	Aufwändiger, braucht Serverunterstützung

Aufbau, Zusammenarbeit und Konfiguration einer dynamischen Website

Aufbau

Dynamische Webseite



Zusammenarbeit

HTTP (Hypertext Transfer Protocol)

HTTP ist ein **Protokoll**, das regelt, wie **Daten zwischen Browser (Client) und Webserver** übertragen werden.

Der Browser schickt eine **HTTP-Anfrage** (Bsp.: GET index.php) an den Webserver

Der Server antwortet mit einer **HTTP-Antwort** (Statuscode + Inhalt wie HTML)

Client (Browser)

Client ist der **Computer des Nutzers,** meistens in Form eines **Browsers** (z. B.: Chrome)

Der Browser schickt http-Anfragen an den Server, wenn man eine Website **aufruft** (z. B. durch Eingabe einer URL).

Webserver, HTTPD (Hypertext Transfer Protocol Daemon), (z. B.: Apache)

-HTTPD, Läuft im Hintergrund in das Webserver

-Empfängt **HTTP-Anfragen** von **Nutzern (Browser)**

-Prüft, ob die angeforderte Datei **statisch (HTML, CSS) oder dynamisch (PHP) ist**

-Gibt **statische Dateien** (HTML, CSS) direkt aus

-Leitet **dynamische Anfragen** (z. B. .php) an den **PHP-Interpreter** weiter.

PHP-Interpreter

-Führt **PHP-Code** aus, (der z. B. **Formulare verarbeitet** oder **Datenbankabfragen** macht)

-Generiert am Ende **dynamisch HTML**, das dann an den **Webserver zurückgegeben** wird.

Datenbank (z. B.: MySQL)

-Speichert **Daten** wie z. B.: Benutzerdaten, Produkte, Beiträge usw.

-PHP verbindet sich mit der **Datenbank, liest** oder **speichert Daten.**

Konfiguration

Um **HTTPD** (Bsp.: Apache) zu **konfigurieren** benutzt man **die Config-Datei** Es ist eine Textdatei, in der man den Webserver einstellt

Wichtige Dinge, die darin stehen können:

-**DocumentRoot**: Welcher Ordner im Dateisystem ist das **„Startverzeichnis"** der Website? (z. B. htdocs)

-Welche **Dateitypen** sollen wie behandelt werden? (.html direkt, .php über PHP)

-**Ports** (z. B. Port 80 für HTTP)

grundlegende php Befehle

Syntax

PHP-Code fängt mit **<?php** an und endet mit **?>**

Am Ende einer **ausführbaren Code-Zeile** muss ein **Semikolon (;)** stehen.

Variablen

-Eine **Variable** ist ein **„Behälter"**, in dem man **Daten speichern** kann

-Alle **Variablennamen** in PHP fangen mit **$** an

-PHP ist dynamisch typisiert
d. H.: **man muss nicht die Dateityp (varchar, int, usw.) eingeben**

```
$name = „Ömer";
$nummer = 25;
```

Operatoren

Operatoren sind Zeichen, mit denen man **Rechnungen macht**, **Vergleiche anstellt** oder **Bedingungen verknüpft**.

Operator	Name	Beispiel
=, +=, -=, *=, /=	Zuweisungsoperatoren	$name = „Ömer“ $name → „Ömer“ $nummer += 2 → $nummer + 2
+,-,*,/	Arithmetische Operatoren	6*2 →12
%	Modulo (gibt den Rest von Division)	5%2 →1
==, <, <=, >, >=, !=	Vergleichsoperatoren	5 == 7 → False 9 == 9 →True 4 < 9 → True 5 >= 5 →True 2 != 3 → True
&&	und	False && False → False True && False → False True && True →True
||	oder	False || False → False True || False → True True || True → True
!	not	!False → True !True → False

If, else if, else

If-Statement prüft, ob ein Bedingung wahr (true) ist und führt dann den Code im Block aus

Else If-Statement wird benutzt wenn es mehrere Bedingungen gibt

Else-Statement wird benutzt wenn keine der Bedingungen wahr (true) sind

```
if (Bedingung) {
// Code, der ausgeführt wird
}
```

```
if ($wetter == "regnerisch") {
   echo "Nimm einen Regenschirm!";
} else if ($wetter == "kalt") {
   echo "Zieh eine Jacke an!";
} else if ($wetter == "sonnig") {
   echo "Sonnenbrille nicht vergessen!";
} else {
   echo "Unbekanntes Wetter...";
}
```

For-Loop

Ein **for-Schleife** führt etwas mehrmals aus

```
for (Start, Bedingung, Schritt) {
    // Code, der wiederholt wird
}
```

Start – wird nur einmal am Anfang ausgeführt (z. B. eine Zählvariable setzen).

Bedingung – solange sie wahr (true) ist, wird der Code im Block ausgeführt

Schritt – wird am Ende jedes Durchlaufs ausgeführt (z. B. Zähler erhöhen)

```
for ($i = 0, $i < 6, $i++) {
    echo "$i";
}
```

Extra: Wichtige Funktionen

Funktion	Bedeutung
echo	Gibt Text auf der Webseite aus
isset()	Prüft, ob eine Variable existiert
die()	Beendet das Skript sofort
$_POST[]	Daten aus einem HTML-Formular holen
$_GET[]	Daten aus der URL holen
$conn = new mysqli(host, username, password, database)	Datenbankverbindung einstellen
$conn->query(...)	SQL-Abfrage ausführen
$conn->prepare(...)	Sichere SQL-Abfrage vorbereiten
$stmt->bind_param(...)	Daten in SQL-Abfrage einsetzen
$stmt->execute()	Abfrage ausführen
$result->fetch_assoc()	Ergebnis als Array holen